

stratified_undersampler

Implementation of the history-preserving stratified random undersampling strategy.

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

StratifiedRandomUndersampler

```
StratifiedRandomUndersampler(target_field: str, target_value: TargetType | set[TargetType], fraction: float | None = None, balance: float | None = None, n: int | None = None, seed: int | None = None)
```

Bases: [BaseSampler](#)

History-preserving stratified random undersampling strategy.

Constructor for StratifiedRandomUndersampler.

The random undersampling is performed according to one (and only one) of the following options to be specified: `fraction`, `balance`, `n`.

Note

It is not guaranteed to provide *exactly* the specified fraction/balance/n.

Parameters:

Name	Type	Description	Default
<code>target_field</code>	<code>str</code>	Name of the field used for stratification.	<i>required</i>
<code>target_value</code>	<code>(bool, int, str or set)</code>	Name of class associated with the positive class in <code>target_field</code> . This class will be maintained complete. It can be a set of classes to represent that all those classes should be kept complete without sampling.	<i>required</i>
<code>fraction</code>	<code>float</code>	Specifies the fraction of the negative class to be sampled. All instances from the positive class will be kept.	<code>None</code>
<code>balance</code>	<code>float</code>	Specifies the desired target balance. E.g. 0.9 means that the positive:negative ratio will be 1:9. All instances from the positive class will be kept, while the negative class will be sampled in order to result in the specified balance ratio.	<code>None</code>
<code>n</code>	<code>int</code>	Specifies the number of instances to sample from the negative class. All instances from the positive class will be kept.	<code>None</code>

Name	Type	Description	Default
<code>seed</code> ↗	<code>int</code>	Random seed for Spark's random number generator.	<code>None</code>

See Also

`pyspark.sql.functions.rand`

`TIMESTAMP_MS_FIELD` `class-attribute` `instance-attribute` [↗](#)

```
TIMESTAMP_MS_FIELD = 'fdz_timestamp_ms'
```

`seed` `property` [↗](#)

```
seed: int
```

`sample` [↗](#)

```
sample(df: _TDataFrame, entity_field: str | None = None, filter_expr: Column | None = None, repartition: bool = False, mark_field: str = MARK_FIELD, optimal_recovery: bool = True, exclude_kick_off_window: bool = True, timestamp_field: str | None = None, recovery_days: int | None = None, entity_fields: list[str] | None = None) -> _TDataFrame
```

Undersample the input data via a history-preserving stratified strategy.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗	<code>DataFrame</code>	Input data, as a pyspark DataFrame.	<i>required</i>
<code>entity_field</code> ↗	<code>str</code>	Name of the field for which recovery should occur. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can only be provided if 'entity_fields' is None and vice-versa.	<code>None</code>
<code>filter_expr</code> ↗	<code>Column</code>	PySpark expression to filter events from the random sampling selection. A typical example is a filter for scorable events.	<code>None</code>
<code>repartition</code> ↗	<code>bool</code>	Indicate if the output DataFrame is to be repartitioned to match the number of instances per partition of the original dataset. Default is False, meaning default partitioning.	<code>False</code>
<code>mark_field</code> ↗	<code>str</code>	Name of the field that will be created to preserve the sampling history. Events that were randomly sampled will have a value 'True' for this field. Only applicable if <code>entity_field</code> is given. Default is "fdz_sample_ismarked".	<code>MARK_FIELD</code>
<code>optimal_recovery</code> ↗	<code>bool</code>	Indicates whether optimal recovery should be performed. If set to False, full recovery will be used instead. Only applicable if <code>entity_field</code> is given. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory.	<code>True</code>
<code>exclude_kick_off_window</code> ↗	<code>bool</code>	Indicates whether first time window should be excluded from marking. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory. This is typically useful because profile metrics are not correctly computed in this window.	<code>True</code>

Name	Type	Description	Default
<code>timestamp_field</code> ¶	<code>str</code> <code>None</code>	Name of the field that will be used to as timestamp. Required if <code>optimal_recovery</code> is <code>True</code> or <code>exclude_kickoff_window</code> is <code>True</code> .	<code>None</code>
<code>recovery_days</code> ¶	<code>int</code> <code>None</code>	Size in days of the window considered. Typically, this value should correspond to the maximum window size of your plan's metrics operator. Applicable if <code>optimal_recovery</code> is <code>True</code> or <code>exclude_kickoff_window</code> is <code>True</code> . In the case that recovery day values are provided through 'entity_fields', this argument is ignored.	<code>None</code>
<code>entity_fields</code> ¶	<code>list</code>	List of field names for which recovery should occur. Alternatively, can be a list of tuples (field, recovery_days) if different windows are to be applied to each of the fields. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can only be provided if 'entity_field' is <code>None</code> and vice-versa. In the case that a list of tuples is provided, 'recovery_days' is ignored. If 'exclude_kickoff_window' is 'True' then the corresponding window will be the maximum value of the recovery_days provided in the 'entity_fields' tuples.	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	Sampled data.

Examples:

This examples demonstrates the behaviour of the option `exclude_kickoff_window`.

```
>>> first_timestamp = df.agg(F.min(timestamp_field)).first()[0]
>>> datetime.fromtimestamp(first_timestamp/1000)
2018-12-20 09:27:53
>>> window_size = 7 #days
>>> df_sampled = sampler.sample(
...     df,
...     mark_field='ismarked',
...     exclude_kickoff_window = True,
...     timestamp_field='timestamp',
...     recovery_days=window_size,
... )
>>> first_marked_timestamp = (
...     df.filter(col(is_marked)).
...     agg(F.min(timestamp_field)).first()[0]
... )
>>> datetime.fromtimestamp(first_marked_timestamp/1000)
2018-12-27 10:15:48
>>> MS_PER_DAY = 24*60*60*1000
>>> first_marked_timestamp >= first_timestamp + max_window*MS_PER_DAY
True
```